Lectura de cartas y control por gestos para Blackjack con visión por computadora: de datos sintéticos a una mesa real

Cristhian Pettico, Rodolfo Di Chiazza, Julian Blanco
Carrera de Especialización en Inteligencia Artificial (CEIA)
Facultad de Ingeniería, Universidad de Buenos Aires
Buenos Aires, Argentina

***Resumen*—En este trabajo se presenta un prototipo que percibe y comprende el juego Blackjack sobre una mesa real. Una cámara reconoce las cartas de cada participante y entiende los gestos realizados por el jugador: pedir carta o plantarse. La estrategia para el reconocimiento del índice de la esquina de la carta fue seleccionada entre dos: (A) un detector YOLO11 de 13 clases y (B) una arquitectura compuesta por un detector de una única clase seguido de una ResNet-18. Ambas alcanzaron un mAP50 $\approx$ 0,99 en el dataset sintético de entrenamiento. A pesar de esto, sobre fotos reales caen a 0,68–0,86. Se realizó fine-tuning con imágenes de otro dataset pertenecientes a otro mazo. El cambio no resultó contundente. Por otro lado, al utilizar 79 fotografías del escenario real, el recall de la arquitectura A subió de 0,66 a 0,96. En términos globales, la estrategia A consiguió un mejor desempeño en la detección y clasificación conjuntas, así como un menor tiempo de procesamiento. Adicionalmente, se evaluó la robustez ante efectos de rotación, oclusión, iluminación y variaciones de fondo. Se concluyó que la oclusión es la perturbación más dañina, la iluminación afecta mucho más en datos reales que en sintéticos y el fondo casi no fue influyente. El prototipo desarrollado resultó ser capaz de seguir y resolver correctamente las cuatro partidas reales grabadas, a 7–10 cuadros por segundo en CPU.**

***Index Terms*—detección de índices, cartas, YOLO11, comparación de modelos, seguimiento, detección de gestos, Blackjack**

I. Motivación

En los casinos, las mesas de juego son supervisadas con cámaras para registrar los movimientos de las manos de los jugadores, detectar errores del crupier y conductas extrañas como el conteo de cartas. Automatizar esa lectura es un problema clásico de visión por computadora. Zutis y Hoey [1] monitorearon partidas de Blackjack con una cámara estéreo cenital usando descriptores clásicos. Trabajos recientes integran el reconocimiento de cartas con la recomendación de la estrategia básica [2] o entrenan detectores YOLO sobre datasets sintéticos y reales de cartas [3]. El problema es, en apariencia simple. Sin embargo, en una mesa real las cartas aparecen giradas, superpuestas, tapadas por las manos y bajo iluminación variable, lo que complejiza la tarea.

El **objetivo** de este trabajo es entrenar un detector de cartas y **evaluar experimentalmente su desempeño** ante rotaciones, oclusiones, cambios de iluminación y de fondo, e integrarlo en una aplicación que permita llevar una partida de Blackjack en una mesa real. Cabe aclarar, que como en Blackjack el palo no influye en el puntaje, el reconocimiento se limita a los 13 valores, los valores numéricos de cada carta.

Las principales contribuciones de este trabajo son:

- Una comparación entre 2 arquitecturas diferentes: un detector simple (A) y una arquitectura basada en dos etapas, un detector y un clasificador (B). Se las comparó en las métricas de precisión y latencia sobre datos sintéticos, un dataset real público y un dataset real propio.
- La cuantificación de la brecha sintético/real y del momento en el que el *fine-tuning* con fotos reales la achica.
- Un estudio de robustez ante 5 tipos de alteraciones de las imágenes y un análisis de los errores entre valores.
- Un sistema completo y reproducible validado con partidas reales.

II. Metodología

La Fig. 1 resume la arquitectura del sistema.

II-A. Datos

Sintético. Se utilizaron 10.100 imágenes de 512×512 píxeles derivadas del dataset *Playing Cards* de Roboflow [4], compuesto por cartas generadas sobre distintos fondos texturados. El conjunto se dividió en 7.070 imágenes de entrenamiento, 2.020 de validación y 1.010 de prueba. Las etiquetas originales se redujeron a los 13 clases de cartas para la arquitectura A y a una única clase, correspondiente al índice de la esquina, para el detector de la arquitectura B. Para entrenar la CNN de B, se recortaron las regiones anotadas añadiendo un margen del 10 %, obteniéndose 28.280 / 8.080 / 4.040 recortes para entrenamiento, validación y prueba, respectivamente.

Real público. Se utilizaron 98 imágenes reales de 416×416 píxeles provenientes de [3], divididas en 69 / 18 / 11 imágenes para entrenamiento, validación y prueba. El dataset contiene cartas de un único palo (corazones) pertenecientes a un mazo cuyos índices se encuentran impresos en las cuatro esquinas.

Real propio. Para representar las condiciones de despliegue del prototipo se grabaron distintas sesiones utilizando la cámara empleada durante el juego. De estas grabaciones se extrajeron 130 cuadros, divididos en 79 / 21 / 30 para entrenamiento, validación y prueba. Las imágenes fueron pre-etiquetadas utilizando la arquitectura A y posteriormente corregidas de forma manual, obteniéndose un total de 1.688

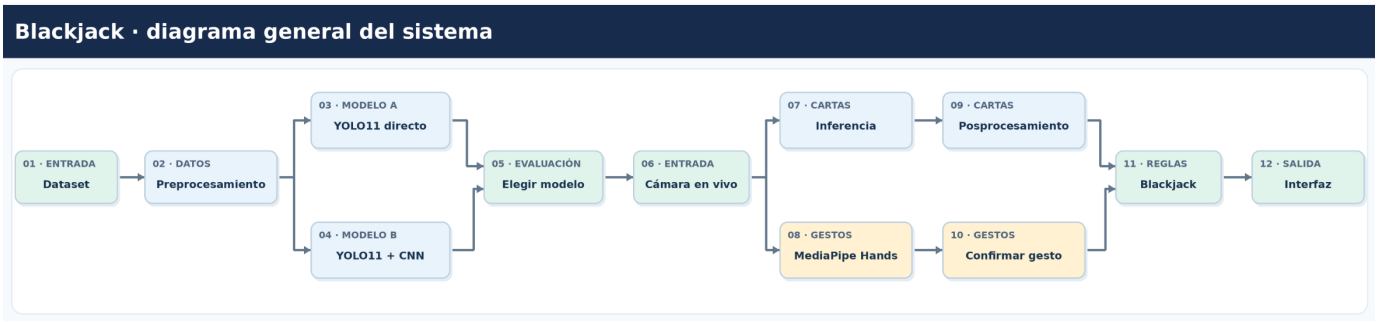


Figura 1: Arquitectura del sistema. Se entrenan los modelos A y B y se elige uno (bloques 01–05); cada cuadro de la cámara pasa por la inferencia de cartas y por MediaPipe, y ambas salidas alimentan el motor de reglas y la interfaz (bloques 06–12).

esquinas anotadas. El conjunto incluye situaciones con múltiples cartas superpuestas y condiciones propias del escenario real de uso. La separación entre entrenamiento, validación y prueba se realizó por sesión, evitando que cuadros consecutivos o visualmente muy similares aparecieran en particiones diferentes y reduciendo así el riesgo de *data leakage*.

II-B. Modelos

A (una etapa): se utilizó YOLO11s [5], [6], inicializado con pesos preentrenados sobre COCO y adaptado para detectar y clasificar directamente los índices de las cartas en 13 clases. Se seleccionó YOLO11s por ofrecer un compromiso adecuado entre capacidad de detección y costo computacional, además de su integración con herramientas de seguimiento utilizadas posteriormente en el procesamiento de video.

B (dos etapas): de manera análoga, se entrenó YOLO11s para la detección de una única clase, correspondiente a los índices localizados en las esquinas de las cartas. Cada región detectada es recortada y enviada a una ResNet-18 [7], inicializada con pesos preentrenados sobre ImageNet y adaptada para clasificar los recortes entre los 13 rangos posibles. Las imágenes de entrada al clasificador se redimensionan a 96×96 píxeles.

La hipótesis que impulsó el desarrollo de ambas arquitecturas fue la idea de que separar "dónde" de "qué" facilita la tarea de identificación de las cartas. Una estrategia conceptualmente similar fue evaluada por Chung y Ravella [8], quienes desacoplaron la localización de las cartas de su clasificación y obtuvieron mejores resultados que con un YOLOv8 entrenado para resolver ambas tareas conjuntamente.

II-C. Entrenamiento

Detectores YOLO. Ambos detectores parten de YOLO11s preentrenado en COCO y se entrenan hasta 50 épocas con imágenes de 640×640 píxeles, lote de 16 y *early stopping* con paciencia de 15 épocas.

Clasificador ResNet-18. Se reemplaza la última capa por una de 13 salidas y se ajusta toda la red (lote de 128, hasta 12 épocas) con AdamW y OneCycleLR (tasa máxima 10^{-3}, *weight decay* 10^{-4}), *label smoothing* de 0,05 y aumentos de rotación ($\pm 15°$), traslación, escala, color y desenfoque. No se usan volteos, porque un índice espejado no es una carta válida. En todos los casos el mejor modelo se elige con la validación y el test se reserva para la evaluación final.

***Fine-tuning* utilizando imágenes reales:** partiendo de los modelos entrenados sobre el dataset sintético, se realizó una segunda etapa de entrenamiento incorporando imágenes reales. Se llevaron a cabo dos experimentos independientes: uno utilizando el dataset real público (*ft*) y otro utilizando el dataset real propio (*propio*). En cada época, las imágenes reales de entrenamiento se combinaron con una muestra de 400 imágenes del dataset sintético, repitiendo cinco veces las imágenes reales para aumentar su representación durante el ajuste. La selección del mejor modelo se realizó utilizando exclusivamente el conjunto de validación real correspondiente, manteniendo el conjunto de test real reservado para la evaluación final.

II-D. De esquinas a cartas y a la partida

Emparejamiento. Cada carta suele generar dos detecciones, correspondientes a dos de sus esquinas opuestas. Para determinar cuáles pertenecen a una misma carta, se comparan la distancia entre sus centroides y la orientación de la diagonal que los une. Dos esquinas se consideran compatibles cuando la distancia entre sus centros, normalizada por el lado mayor de las cajas detectadas, se encuentra entre 3,3 y 4,8, y la línea que las une presenta un ángulo próximo a $50°$ para cartas verticales o a $140°$ para cartas giradas. La utilización del lado mayor como referencia permite mantener una medida consistente ante cambios en la orientación de los índices. Estos umbrales se calibraron empíricamente a partir de las esquinas etiquetadas en el dataset propio.

Entre los pares compatibles se priorizan aquellos con mejor ajuste geométrico, evitando asignar una misma esquina a más de una carta. Si una esquina no puede emparejarse, se conserva igualmente como una carta individual, asumiendo que su esquina opuesta puede encontrarse ocluida. Finalmente, se eliminan detecciones redundantes. Por ejemplo, si el modelo detecta más de una vez una misma esquina, se conserva únicamente una de las detecciones. Asimismo, una detección adicional situada dentro de una carta ya reconstruida se descarta cuando su geometría no resulta compatible con la existencia de una carta diferente.

Seguimiento. Para reducir las fluctuaciones entre cuadros consecutivos se utiliza ByteTrack [9], que asigna un identificador persistente (*track ID*) a cada esquina detectada. Para cada identificador, el valor de la carta se estabiliza mediante una votación ponderada por confianza sobre las detecciones de los últimos 15 cuadros. Además, si una esquina deja de detectarse temporalmente, su última posición y valor se conservan durante un máximo de 10 cuadros, permitiendo tolerar pérdidas breves producidas por oclusiones, reflejos o desenfoque. Finalmente, la mano mostrada se estabiliza considerando los últimos 15 cuadros y tomando, para cada valor de carta, la mediana de la cantidad de apariciones observadas.

Gestos. Para la interacción del jugador se utiliza MediaPipe Hands [10], que proporciona 21 puntos de referencia (*landmarks*) por mano. A partir de ellos, la pose se clasifica utilizando cocientes entre las distancias muñeca–punta y muñeca–nudillo de los dedos, lo que permite reducir la dependencia respecto de la escala y orientación de la mano. El gesto de *apuntar*, definido por el índice extendido y sostenido durante al menos 0,5 s, solicita una nueva carta. Por su parte, una *palma abierta*, sostenida durante al menos 1,2 s o acompañada por un desplazamiento, indica que el jugador desea plantarse.

Reglas. La lógica del juego se implementa mediante un motor de estados que distingue las etapas de reparto, turno del jugador, turno de la casa y fin de la partida. La mano estabilizada por el módulo de seguimiento constituye la entrada válida para actualizar el estado del juego. La casa solicita cartas mientras su puntuación sea inferior a 17, y el valor del As se ajusta dinámicamente entre 11 y 1 para evitar superar 21 cuando sea posible. Para evitar que fluctuaciones transitorias en la detección modifiquen el estado de la partida, los cambios en la mano deben persistir durante varios cuadros antes de ser aceptados. Se requieren 5 cuadros estables para cambios esperados y 20 cuando aparecen cartas que no fueron solicitadas. Además, mientras una mano permanece sobre la mesa no se confirma un nuevo cambio de estado.

II-E. Protocolo de evaluación

Una detección se considera correctamente localizada cuando presenta una intersección sobre unión (IoU, *Intersection over Union*) mayor o igual a $0{,}5$ con una esquina anotada. Para considerar correcta la detección completa, además debe coincidir el rango predicho con el rango real. Se reportan mAP@0.5, precisión (P), *recall* (R) y F1, junto con la latencia media de inferencia por imagen. Para las métricas basadas en predicciones se utiliza un umbral de confianza de $0{,}5$. La latencia se mide en CPU sobre un AMD Ryzen 7 7840U.

Robustez. La robustez de los modelos se evalúa sobre el conjunto de test propio (30 cuadros, 412 esquinas anotadas) y sobre una muestra fija de 100 imágenes del test sintético. Sobre estas imágenes se aplican cinco familias de perturbaciones visuales (Fig. 2): rotación, oclusión, variaciones de brillo, variaciones de contraste y sombras. La rotación se realiza alrededor del centro de la imagen, con seis niveles entre $15°$ y $180°$; las anotaciones se transforman junto con la imagen.

Tabla I: Comparación de las arquitecturas A y B en los dominios sintético y reales, antes y después del *fine-tuning*. *ft*: ajuste con el dataset real público; *propio*: ajuste con el dataset propio.

	Sintético		Real público		Real propio	
Modelo	mAP50	F1	mAP50	F1	mAP50	F1
A base	0,995	1,00	0,68	0,72	0,86	0,78
B base	0,995†	1,00	0,68	0,72	0,82	0,78
A *ft*	–	–	**0,89**	**0,89**	0,86	0,78
B *ft*	–	–	0,77	0,78	0,90	0,83
A *propio*	–	–	0,70	0,71	**0,98**	**0,96**
B *propio*	–	–	0,75	0,73	0,94	0,93
Latencia (ms/imagen): A 100–114; B 136–143						

†Detector de B; la CNN alcanza 100 % de exactitud en el test sintético.

La oclusión se simula cubriendo cada índice mediante un rectángulo generado desde uno de sus bordes, con seis niveles de severidad entre el 10 % y el 60 %. El brillo se modifica mediante seis factores de ganancia entre $\times 0{,}2$ y $\times 2{,}5$. El contraste se evalúa en cuatro niveles y las sombras se simulan mediante un gradiente de iluminación, también en cuatro niveles de severidad.

Para garantizar una comparación controlada, todos los modelos reciben exactamente las mismas imágenes perturbadas, utilizando una semilla fija para las transformaciones aleatorias. La robustez frente al cambio de fondo se evalúa por separado. Dado que reemplazar el fondo de fotografías reales mediante una máscara de color no produjo composiciones suficientemente confiables, se generaron 30 escenas sintéticas a partir de siete cartas reales recortadas del mazo utilizado en la demostración. Las escenas contienen seis rangos y un total de 244 esquinas anotadas, distribuidas sobre ocho fondos diferentes. Las cartas y sus posiciones se mantienen constantes entre configuraciones, de modo que únicamente varía el fondo.

Análisis de errores. Para caracterizar los errores de detección y clasificación, cada esquina real se asocia con la predicción que presenta la mayor superposición espacial, siempre que su IoU sea mayor o igual a $0{,}5$, independientemente del rango predicho. Esto permite distinguir entre errores de clasificación (la esquina se localiza correctamente pero se asigna un rango incorrecto), falsos negativos (la esquina no es detectada) y falsos positivos (detecciones que no pueden asociarse con ninguna esquina real).

III. Resultados

III-A. Brecha sintético–real y fine-tuning

La Tabla I evidencia una marcada brecha entre el dominio sintético y los escenarios reales. En el dominio sintético, ambas arquitecturas alcanzan un mAP50 de 0,995 en validación y un F1 de 1,00 sobre el test; adicionalmente, la CNN de la arquitectura B obtiene una exactitud del 100 % sobre los recortes sintéticos. Sin embargo, al evaluar los modelos base sobre imágenes reales, el mAP50 disminuye hasta valores comprendidos entre 0,68 y 0,86. Este resultado muestra que un desempeño prácticamente perfecto sobre datos sintéticos

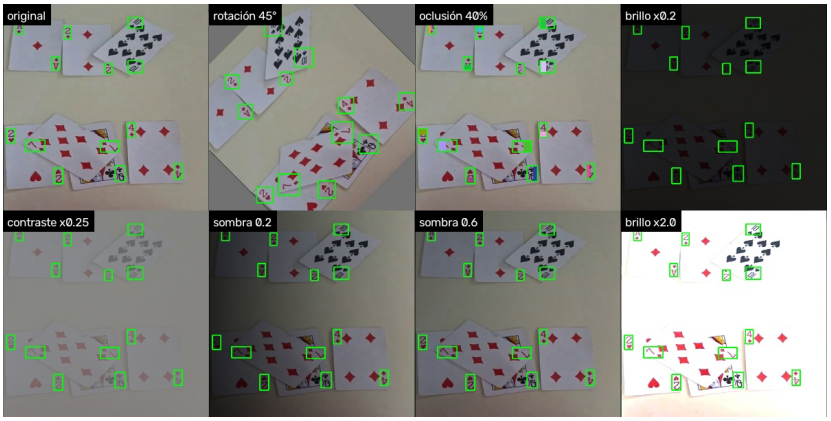


Figura 2: Ejemplos de las cinco familias de perturbaciones utilizadas para evaluar robustez: rotación, oclusión, brillo, contraste y sombra. Las cajas verdes indican las esquinas anotadas. Cada familia se evalúa en distintos niveles de severidad; en las sombras, el valor indicado corresponde al nivel mínimo de iluminación.

no garantiza una generalización equivalente ante cambios de mazo, iluminación, fondo, orientación y condiciones de captura.

El *fine-tuning* con imágenes reales permite reducir esta brecha, aunque su efecto depende fuertemente del dominio utilizado para el ajuste. La arquitectura A ajustada con el dataset real público aumenta su F1 de 0,72 a 0,89 sobre ese mismo dominio, pero mantiene un F1 de 0,78 sobre el escenario propio. De manera análoga, al realizar el ajuste con las 79 imágenes de entrenamiento del dataset propio, A alcanza un mAP50 de 0,98 y un F1 de 0,96 en dicho escenario, mientras que su desempeño sobre el dataset público permanece próximo al del modelo base.

Este comportamiento indica que la adaptación con una cantidad reducida de imágenes representativas del escenario de despliegue resulta más efectiva que utilizar imágenes reales provenientes de otro mazo y otras condiciones de captura. La arquitectura B presenta una transferencia ligeramente mayor entre dominios: al ajustarla con el dataset público, su F1 sobre el escenario propio aumenta de 0,78 a 0,83. Una posible explicación es que parte del ajuste afecta al detector de una única clase, cuya tarea consiste únicamente en localizar índices y no en reconocer directamente su valor. No obstante, esta mejora continúa siendo inferior a la obtenida al incorporar imágenes del escenario objetivo, donde B alcanza un F1 de 0,93.

Selección del modelo. A partir de estos resultados se seleccionó la arquitectura A para la aplicación final. Una vez ajustada con imágenes propias, A obtiene el mejor desempeño en el dominio de despliegue, con mAP50 de 0,98 y F1 de 0,96, frente a 0,94 y 0,93 de B, respectivamente. La diferencia también se observa en el costo computacional: A requiere entre 100 y 114 ms por imagen, mientras que B requiere entre 136 y 143 ms.

Además de la mejora cuantitativa, A presenta una arquitectura más simple para la operación en tiempo real. Cada índice es localizado y clasificado mediante una única inferencia, mientras que B requiere primero ejecutar YOLO para localizarlo y posteriormente procesar cada recorte mediante ResNet-18. Esta segunda estrategia introduce una etapa adicional de cómputo y una posible propagación de errores: una localización imprecisa del detector puede producir un recorte incompleto o descentrado y afectar posteriormente la clasificación. Por lo tanto, dado que la separación entre localización y clasificación no produjo una mejora de precisión que compensara este costo adicional, se optó por A como modelo final por ofrecer el mejor compromiso entre desempeño, latencia y simplicidad de integración.

Tabla II: F1 medio de cada arquitectura ante las distintas familias de perturbaciones. Cada valor corresponde al promedio sobre los niveles de severidad evaluados. La categoría iluminación agrupa las perturbaciones de brillo, contraste y sombra.

Dominio	Familia	A	B	A *propio*	B *propio*
Real propio	Rotación	0,76	0,72	**0,81**	0,80
	Oclusión	0,28	0,22	**0,52**	0,46
	Iluminación	0,69	0,58	**0,88**	0,82
Sintético	Rotación	0,74	0,72	0,74	0,71
	Oclusión	**0,74**	0,63	0,72	0,61
	Iluminación	**0,99**	0,98	0,99	0,98

Tabla III: F1 sobre escenas compuestas manteniendo las mismas siete cartas y posiciones y variando únicamente el fondo.

	Mad.	Verde	Osc.	Blanco	Text.	Desor.	Ruido	Márm.
A	0,99	1,00	1,00	0,99	1,00	1,00	0,99	1,00
B	0,98	0,99	0,99	**0,90**	0,97	0,97	0,98	0,99
A *propio*	0,99	0,99	0,99	0,98	0,99	0,99	0,99	0,99
B *propio*	1,00	1,00	1,00	1,00	1,00	1,00	0,99	1,00

III-B. Robustez

La Tabla II resume cada familia y la Fig. 3 muestra la evolución del F1 con la severidad.

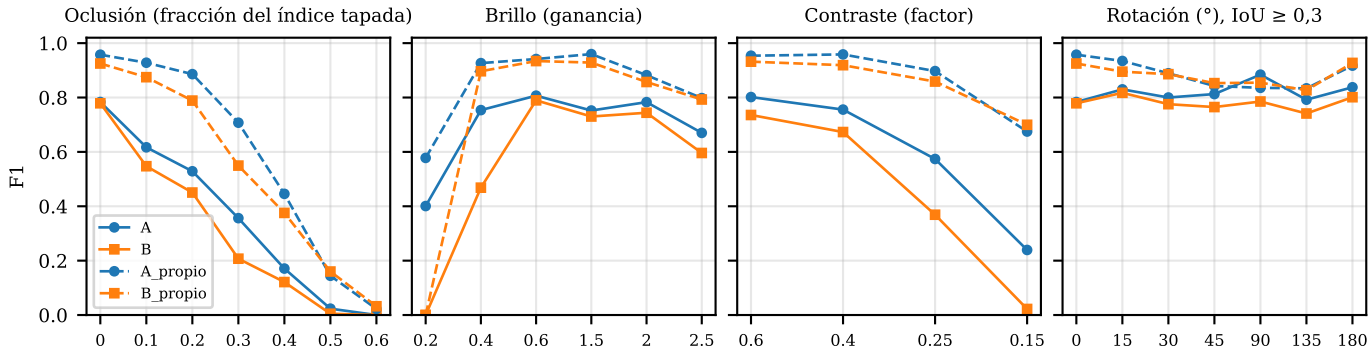


Figura 3: F1 en el test propio en función de la severidad de cada perturbación (trazo lleno: modelos base; trazo cortado: ajustados con fotos propias). Las rotaciones se miden con IoU $\geq$ 0,3 (ver texto).

Oclusión. La oclusión produce la mayor degradación entre las perturbaciones evaluadas. En el dominio sintético, A mantiene un F1 de 0,95 con un 30 % del índice ocluido y desciende a 0,52 al alcanzar el 50 %. La degradación aparece considerablemente antes en imágenes reales: A base cae de 0,78 a 0,62 con apenas un 10 % de oclusión y a 0,36 con un 30 %. El *fine-tuning* con imágenes propias aumenta notablemente esta tolerancia, alcanzando F1 de 0,93 y 0,89 para oclusiones del 10 % y 20 %, respectivamente. Este resultado motivó decisiones posteriores del sistema, como conservar esquinas no emparejadas como cartas válidas y mantener temporalmente detecciones perdidas mediante el módulo de seguimiento.

Iluminación. Se observa una diferencia marcada entre los dominios sintético y real. Sobre imágenes sintéticas, las perturbaciones de iluminación apenas modifican el desempeño medio, que permanece por encima de 0,98. En cambio, las imágenes reales presentan una sensibilidad considerable a condiciones extremas de subexposición y bajo contraste. Con brillo reducido a $\times 0,2$, A base alcanza F1 de 0,40 y B deja de producir detecciones correctas; con contraste $\times 0,15$, A base desciende a 0,24. El ajuste con imágenes propias mejora especialmente a A, cuyo F1 medio frente a perturbaciones de iluminación aumenta de 0,69 a 0,88. Parte de la brecha sintético/real se debe, entonces, a la iluminación del escenario.

Rotación. La evaluación de rotaciones requiere considerar el efecto de la métrica de localización. Al utilizar IoU $\geq 0,5$, se observa una caída pronunciada a $45°$ incluso sobre imágenes sintéticas. Sin embargo, parte de esta disminución se debe a que las cajas anotadas y predichas permanecen alineadas con los ejes, por lo que la caja envolvente de un índice rotado aumenta de tamaño y reduce artificialmente su IoU. Al relajar únicamente para este análisis el umbral a IoU $\geq 0,3$, A alcanza F1 de 0,98 a $45°$ y valores próximos a 1,00 para rotaciones mayores en el dominio sintético. En el dominio real persiste una degradación moderada, lo que indica que no todo el efecto puede atribuirse a la métrica. Una posible explicación adicional es que las imágenes del dataset propio fueron capturadas manteniendo una única orientación de la cámara, por lo que las orientaciones rotadas se encuentran menos representadas durante el ajuste del modelo.

Tabla IV: Errores por esquina en el test propio (412 esquinas reales).

	Bien	Valor equivocado	No detectada	Falsos positivos	Confusión más frecuente
A	66,3 %	1,7 %	**32,0 %**	5	8→6 (5)
B	64,1 %	0,2 %	**35,7 %**	1	A→4 (1)
A *propio*	**95,6 %**	**0 %**	4,4 %	17	ninguna
B *propio*	93,0 %	2,2 %	4,9 %	24	5→3 (5)

Fondo. A diferencia de las perturbaciones anteriores, el cambio de fondo presenta un impacto mínimo. En las escenas controladas de la Tabla III, A mantiene F1 $\geq 0,99$ en los ocho fondos evaluados. El caso de mayor degradación corresponde a B base sobre fondo blanco, con F1 de 0,90. Dado que las cartas y sus posiciones permanecen constantes entre escenas, estos resultados indican que el fondo, por sí solo, no constituye una fuente importante de error para el sistema.

III-C. Análisis de errores

La Tabla IV permite identificar el origen de los errores observados en el test propio. En los modelos base, el principal problema no es la asignación incorrecta del valor, sino la ausencia de detección: A no detecta el 32,0 % de las esquinas y B el 35,7 %, mientras que los errores de clasificación representan únicamente el 1,7 % y 0,2 %, respectivamente.

El *fine-tuning* con imágenes propias modifica sustancialmente este comportamiento. En A *propio*, las esquinas no detectadas disminuyen del 32,0 % al 4,4 % y no se observan errores de clasificación, aunque aumenta el número de falsos positivos de 5 a 17. B presenta una tendencia similar, reduciendo las esquinas no detectadas al 4,9 %, pero conserva un 2,2 % de errores de clasificación y genera 24 falsos positivos.

El error residual se concentra en el rango 8: 13 de las 18 esquinas que A *propio* no detecta, y 11 de las 20 de B *propio*.

III-D. Sistema completo

Seguimiento. Para evaluar el aporte del seguimiento temporal se utilizaron videos con movimiento de cámara simulado. La incorporación del módulo de seguimiento aumentó la

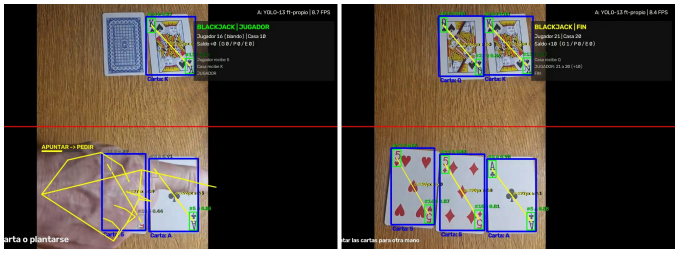


Figura 4: Ejemplo de una partida real procesada por el sistema. Izquierda: el gesto de *apuntar* solicita una nueva carta. Derecha: estado final de la partida, con las cartas detectadas, los puntajes y los eventos registrados por la aplicación.

proporción de cuadros en los que se mostró correctamente la mano del 50,5 % al 76,2 %. Además, en intervalos de 10 s, la cantidad de cambios en la mano mostrada se redujo de 47 a 4. Estos resultados muestran que la combinación del seguimiento de las esquinas, la memoria temporal y la estabilización de la mano reduce considerablemente las fluctuaciones producidas por pérdidas momentáneas de detecciones.

Gestos. El reconocimiento de gestos se evaluó sobre imágenes reales de HaGRID [11]. La estrategia propuesta alcanza un *recall* de 0,80 para el gesto de *apuntar* y de 0,89 para *palma*, manteniendo valores similares al rotar las imágenes 90° y 180°. Como referencia, el reconocedor de gestos preentrenado de MediaPipe utilizado como comparación deja de reconocer correctamente ambos gestos al rotar las imágenes 90° o 180°, alcanzando un *recall* de 0 en estas orientaciones, algo habitual con una cámara cenital. En clips de video, el gesto de *palma* se reconoce correctamente en el 82–87 % de los casos, con un 4,2 % de activaciones falsas. **Partidas reales.** Finalmente, se realizó una validación integral del prototipo mediante partidas completas de Blackjack grabadas en el escenario de la aplicación. Los casos evaluados incluyeron distintas situaciones del juego, entre ellas victorias del jugador y de la Casa, el ajuste dinámico del valor del As de 11 a 1 y la presencia de cartas giradas. Al reproducir posteriormente las partidas cuadro a cuadro, el sistema reprodujo correctamente los resultados finales, las cartas detectadas y los gestos registrados durante la ejecución en vivo (Fig. 4). Las partidas quedan como prueba de regresión del sistema, que opera a 7–10 cuadros por segundo en CPU.

IV. Conclusiones

1. El rendimiento en el dataset sintético (mAP50 0,995) no anticipa el rendimiento real (0,68–0,86): evaluar con fotos del escenario de uso es imprescindible.
2. El *fine-tuning* con datos reales mejora sobre todo el dominio con el que se hace: 79 cuadros del escenario propio llevaron el recall de A de 0,66 a 0,96, mientras que fotos de otro mazo no mejoraron a A y solo mejoraron parcialmente a B (F1 de 0,78 a 0,83), a través de su detector.
3. En el escenario de uso, el detector de una etapa (A) superó al de dos (B) en exactitud y latencia: el contexto que ve el detector ayuda a distinguir valores similares, mientras que la CNN de B clasifica un recorte aislado.
4. En datos reales el error dominante no es confundir valores sino no encontrar la esquina; la oclusión es la perturbación más dañina y la iluminación pesa mucho más que en los datos sintéticos.
5. El fondo casi no influye, y la caída aparente ante rotaciones oblicuas se explica en gran parte por la métrica (IoU de cajas alineadas).
6. La lógica de posprocesamiento —emparejamiento geométrico, supresión de detecciones superpuestas mediante NMS agnóstico a la clase, seguimiento temporal y congelamiento del estado de la mesa ante oclusiones por las manos— fue decisiva para pasar de detecciones independientes por fotograma a una representación consistente de las manos en partidas reales.
7. Para gestos con cámara cenital, reglas invariantes a la rotación sobre los puntos de MediaPipe funcionan donde el clasificador preentrenado falla.

Trabajo futuro: reconocer el palo (permitiría detectar cartas duplicadas, imposibles con un solo mazo), entrenar con la cámara en ambas orientaciones, sumar mesas e iluminaciones al test propio y ampliar el sistema a varios jugadores. El código, los modelos, los datos y las partidas reales están disponibles en https://github.com/dichiazzar/VpC_2_TP_2.

Referencias

[1] K. Zutis and J. Hoey, "Who's counting? Real-time blackjack monitoring for card counting detection," in *Proc. 7th Int. Conf. on Computer Vision Systems (ICVS)*, ser. Lecture Notes in Computer Science, vol. 5815. Liège, Belgium: Springer, 2009.

[2] K. Gupta, D. Bolt, and B. Hinchliff, "Optimal blackjack strategy recommender: A comprehensive study on computer vision integration for enhanced gameplay," arXiv:2404.00191, 2024.

[3] T. Kostadinov, "Playing cards object detection," https://github.com/TeogopK/Playing-Cards-Object-Detection, 2024, proyecto del curso Deep Learning, FMI, Universidad de Sofía; dataset real bajo licencia CC0.

[4] Augmented Startups, "Playing cards dataset," Roboflow Universe, https://universe.roboflow.com/augmented-startups/playing-cards-ow27d, accedido: oct. 2026.

[5] G. Jocher and J. Qiu, "Ultralytics YOLO11," https://github.com/ultralytics/ultralytics, 2024, versión 11.0.0, licencia AGPL-3.0.

[6] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," arXiv:2410.17725, 2024.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[8] A. Chung and P. S. Ravella, "Blackjack card counting," Informe de proyecto, Stanford CS231n, https://cs231n.stanford.edu/2024/papers/blackjack-card-counting.pdf, 2024.

[9] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conf. on Computer Vision (ECCV)*, 2022.

[10] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device real-time hand tracking," arXiv:2006.10214, 2020.

[11] A. Kapitanov, K. Kvanchiani, A. Nagaev, R. Kraynov, and A. Makhliarchuk, "HaGRID – HAnd Gesture Recognition Image Dataset," in *Proc. IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV)*, 2024.